

Seminararbeit

Teilnahme-ID: 76626

Bearbeiter/-in dieser Aufgabe:
Quirin Klemm

7. Oktober 2025

Inhaltsverzeichnis

1	Einleitung	2
2	Aufgabe 1: Schmucknachrichten	2
2.1	Einleitung	2
2.2	Lösungsidee	2
2.2.1	Perlen mit gleichen Durchmessern	2
2.2.2	Perlen mit unterschiedlichen Durchmessern	4
2.3	Umsetzung	5
2.3.1	Erstellung des Baums	5
2.3.2	Verbesserung des Baums	6
2.4	Laufzeit und Diskussion	7
2.4.1	Analyse des Algorithmus	7
2.4.2	Stand der Forschung	8
2.4.3	Fazit	9
2.5	Vergleich zur Lösung	9
3	Aufgabe 2: Simultane Labyrinth	9
3.1	Einleitung	9
3.2	Lösungsidee	9
3.2.1	Erstellung der Matrix	10
3.2.2	Lösen der Labyrinth	10
3.2.3	Gruben	10
3.3	Umsetzung	10
3.3.1	Erstellung der Matrix	11
3.3.2	Lösen der Labyrinth	11
3.4	Laufzeit und Diskussion	14
3.4.1	Komplexität der Lösung	14
3.4.2	Laufzeit	14
3.4.3	Fazit	15
3.5	Vergleich zur Lösung	16
4	Fazit	16
5	Anhang	16
5.1	Beispiele	16
5.2	Quellcode	16
5.3	Quellen	16

1 Einleitung

Die vorliegende Arbeit ist meine Einreichung für die zweiten Runde des 43. Bundeswettbewerb Informatik (BwInf). Ich habe mich dazu entschieden, sie anstelle einer W-Seminararbeit im Leitfach Physik zum Thema Klima abzugeben.

In der zweiten Runde des Wettbewerbs wurden drei Aufgaben gestellt, von denen zwei zu bearbeiten und schriftlich auszuarbeiten werden mussten. Jede Aufgabe beschreibt ein Problem, für welches anschließend ein Algorithmus entwickelt werden soll, mit dem sich dieses lösen lässt. Oft wird die Aufgabe selber noch in Teilaufgaben unterteilt, welche den Schwierigkeitsgrad schrittweise erhöhen. Zusätzlich werden Testfälle gegeben, anhand derer der Algorithmus überprüft werden kann. Die erwarteten Ergebnisse der Testfälle werden jedoch nicht angegeben. Sie dienen ausschließlich dazu, dass die Korrektoren die Richtigkeit und Genauigkeit des Algorithmus beurteilen können. Die ausgearbeiteten Lösungen werden anschließend nach einem festgelegten Schema korrigiert und benotet. Pro Aufgabe können bis zu 20 Punkte erreicht werden, wobei gute Ausarbeitungen Zusatzpunkte geben können und unvollständige Bearbeitungen Punktabzüge nach sich ziehen.

Ich erreichte insgesamt 38 Punkte, erhielt damit den 2. Preis und (gehörte damit zu den besten 60 Teilnehmer).

Meine Arbeit ist in zwei Teile aufgeteilt, beginnend mit Aufgabe 1, Schmucknachrichten, und dann Aufgabe 2, simultane Labyrinth. Beide Teile sind gleich aufgebaut und orientiert sich, abgesehen von der Erweiterung um die Einleitung und dem Vergleich mit der Musterlösung, an der Struktur der Vorlagen des BwInf-Teams.

Zuerst erkläre ich kurz die Aufgabestellung. Danach beschreibe ich meine theoretische Umsetzung und zeige im Anschluss, wie ich sie praktisch umgesetzt habe. Anschließend diskutiere ich meine Ergebnisse, ziehe ein Fazit und vergleiche schließlich meine Lösung mit der offiziellen Musterlösung.

2 Aufgabe 1: Schmucknachrichten

2.1 Einleitung

In dieser Aufgabe möchten Sybille und Pedram kurze Nachrichten, die aus verschiedenen Buchstaben bestehen, in Perlen-Codes für Ketten umwandeln. Für jeden Buchstaben ist ein Codewort aus Perlen zu finden, wobei insgesamt ein präfixfreier Code entstehen muss. Dabei soll die Gesamtlänge der codierten Nachricht möglichst gering sein. Daher wird ein präfixfreier Code gesucht, bei dem häufig vorkommende Buchstaben kurze Codewörter erhalten, während seltenere Buchstaben längere Codewörter zugewiesen bekommen. Ein präfixfreier Code ist ein Code bei dem kein Codewort der Anfang eines Codewortes ist. (vlt. Zitat woher das kommt) Dadurch lässt sich jede Nachricht eindeutig codieren und wieder decodieren. Ein Beispiel für einen solchen Code wäre das Morsealphabet. Zusätzlich wird in dieser Aufgabe zwischen zwei Varianten unterschieden: In der ersten Teilaufgabe haben alle Perlen denselben Durchmesser, während in der zweiten Teilaufgabe die Perlen unterschiedliche Durchmesser besitzen.

Ich bearbeite zuerst die Aufgabe, mit den gleichen Durchmessern, und anschließend mit unterschiedlichen Durchmessern.

2.2 Lösungsidee

2.2.1 Perlen mit gleichen Durchmessern

Im Allgemeinen gilt, solange alle Perlen im Durchmesser gleich sind:

$$c_i = p_i \cdot n_i \tag{1}$$

$$l = \sum_i c_i \tag{2}$$

- c_i die Kosten des Buchstaben i
- p_i die Häufigkeit des Buchstabens i in der Nachricht
- n_i die Länge des Codewortes für den Buchstaben i (Anzahl der Perlen)
- l die Länge der codierten Nachricht

Die Gesamtlänge der Nachricht entspricht der Summe aller Kosten jedes einzelnen Buchstaben.

Die Kosten beschreiben dabei, um wie viel die codierte Nachricht wegen des Buchstabens länger wird. Um einen präfixfreien Code zu erzeugen, habe ich mich entschieden einen Baum zu benutzen. Da durch diesen sicher gestellt werden kann, dass kein Codewort der Anfang eines anderen Codeworts ist. Nach meiner Definition ist jedes Blatt des Baums ein Codewort und jede Kante zum Blatt ein Teil des Codeworts. Der Baum hat mindestens so viele Blätter wie es verschiedene Buchstaben gibt. Jeder innere Knoten hat dabei maximal so viele Kinder, wie es verschiedene Perlen gibt. Die Ebene des Blatts entspricht der Länge des Codeworts, also n .

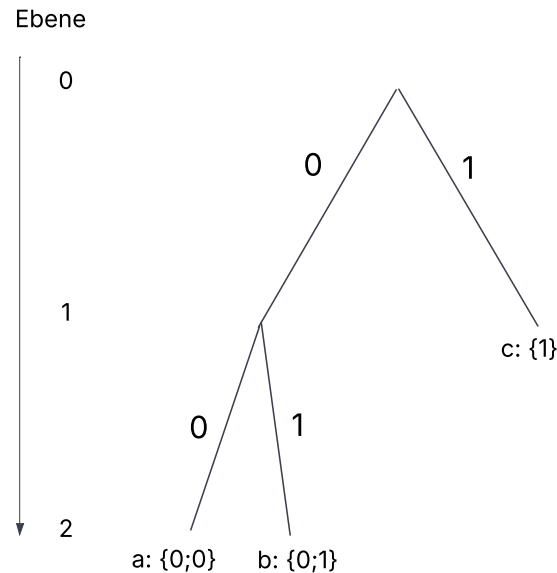


Abbildung 1: simple tree

Um einen möglichst kleinen präfixfreien Code zu erstellen, muss der Buchstabe, der am häufigsten vorkommt, ganz oben eingeordnet werden und die nächsten Buchstaben dann in absteigender Reihenfolge. In Abbildung 1 hätte c die höchste Häufigkeit und danach kommt a und b . Ob der Buchstabe a oder b die höhere Häufigkeit besitzt, ist nicht relevant, da beide auf der untersten Ebene sind.

Daher ergibt sich $p_1 \geq p_2 \geq p_3 \dots \geq p_i$ und $n_1 \leq n_2 \leq n_3 \dots \leq n_i$

Zu finden ist also ein Baum, der basierend auf den Häufigkeiten der Buchstaben, seine Blätter so auf seine Ebenen verteilt, dass die Länge der codierten Nachricht (Gleichung (2)) möglichst gering ist. Jeder innere Knoten muss dabei mindestens zwei Kinder haben, da ein innerer Knoten mit nur einem Kind die Äste des Baums verlängert (höheres n) ohne einen Vorteile zu bringen.

Zu Beginn des Algorithmus wird ein balancierter Baum erstellt, der so viele Blätter wie Buchstaben hat (Abbildung 2). Jedes Blatt wird von links nach rechts in aufsteigender Reihenfolge der Häufigkeit, mit einem Buchstaben belegt, und die Gesamtkosten des Baums werden gemäß der Formel (2) berechnet. Dabei ergeben sich deutlich höhere Kosten für die rechten Buchstaben im Baum im Vergleich zu den linken, da alle Blätter auf derselben Ebene sind, jedoch die Häufigkeit der Buchstaben auf der rechten Seite größer ist. Um dies auszugleichen, wird der Buchstabe mit den höchsten Kosten eine Ebene nach oben verschoben, indem der Elternknoten des Blattes selber zum Blatt wird und alle Kinder verliert. Dadurch verliert der Baum aber zu viele Kinder, weshalb er erweitert werden muss. Für die Erweiterung untersucht der Algorithmus, an welche Knoten neue Blätter angehängt werden können. Er überprüft die Blätter nach ihrer Ebene, d.h. nach ihrem Wert für n , und fügt das mit dem kleinsten Wert hinzu. Das nach oben verschobenene Blatt erhält den Zusatz *protected*. Das verhindert, dass es selbst weitere Kinder bekommen kann. Nach jeder Iteration werden den Blättern wie zu Beginn von links nach rechts Buchstaben aufsteigend hinzugefügt. Das stellt sicher, dass immer die äußerst linken Blätter (d.h. die mit dem höchsten n -Wert), die Buchstaben mit den geringsten Häufigkeiten kriegen und rechts die höchsten. Dadurch wird ein Zyklus vermieden, bei dem ein Blatt zunächst nach oben verschoben und anschließend

wieder nach unten erweitert würde.

Das Verschieben der Blätter nach oben, sowie das Neueinfügen der Blätter unten, gilt als Optimierungsschritt, welcher im Optimierungsprozess wiederholt wird. (Abbildung 4) Dieser Vorgang wird so lange fortgesetzt, bis keine neuen Blätter mehr eingefügt werden können, da alle Blätter als *protected* gelten (Abbildung 5) - also keine weiteren Optimierungen mehr möglich sind. Das ist der Fall kurz vor einem maximal ausgearteten Baum. Der Algorithmus geht also von einem Maximum (maximal ausbalanciert) zum anderen Maximum (maximal ausgeartet). Am Ende wird der Baum mit den geringsten Gesamtkosten ausgegeben.

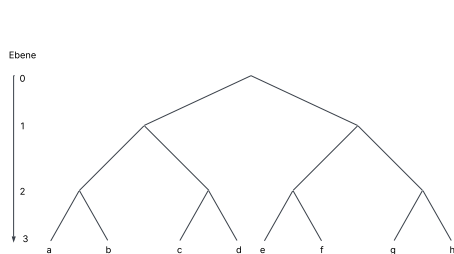


Abbildung 2: Balancierter Baum

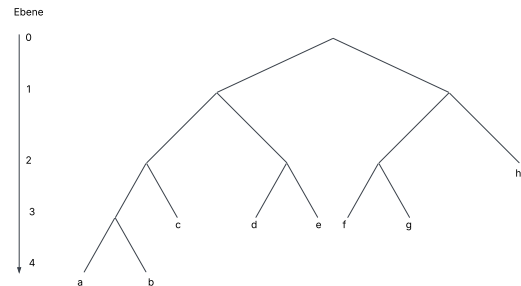


Abbildung 3: Transformation 1

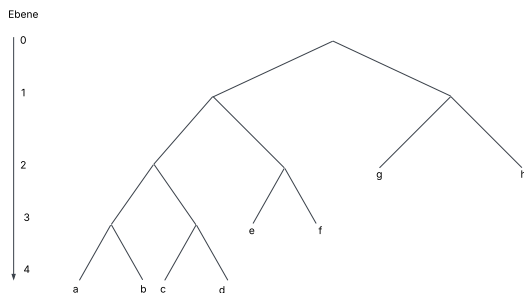


Abbildung 4: Transformation 2

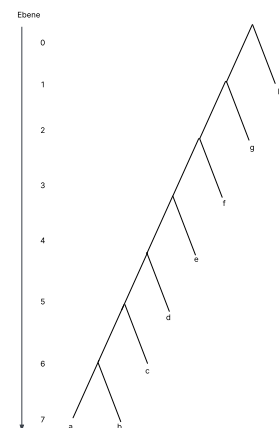


Abbildung 5: Ausgearteter Baum

2.2.2 Perlen mit unterschiedlichen Durchmessern

Das Problem bei unterschiedlichen Durchmessern besteht darin, dass Blätter auf derselben Ebene unterschiedliche Kosten für die Gesamtlänge der Nachricht haben können. Daher sind die Kosten nicht mehr der Wert der Ebene und damit die Anzahl an Perlen. Das hat zu Folge, dass die Gleichung (1) nicht mehr stimmt. Stattdessen muss der Durchmesser jeder Perle im Codewort einzeln angeschaut werden. Daher ergibt sich

$$c_i = p_i \cdot \sum_n cp_n \quad (3)$$

- cp_n die Kosten/Durchmesser der Perle n

Stellt man nun Bäume nicht nach ihrer Ebene, sondern nach ihren Kosten dar, erhält man einen *lopsided tree* [1].

Auch hier gilt: Jeder innere Knoten muss mindestens zwei Kinder haben. Bei dieser Art von Bäumen hat jedoch nicht der äußerst rechte Knoten die geringsten Kosten, sondern der oberste. Der Algorithmus

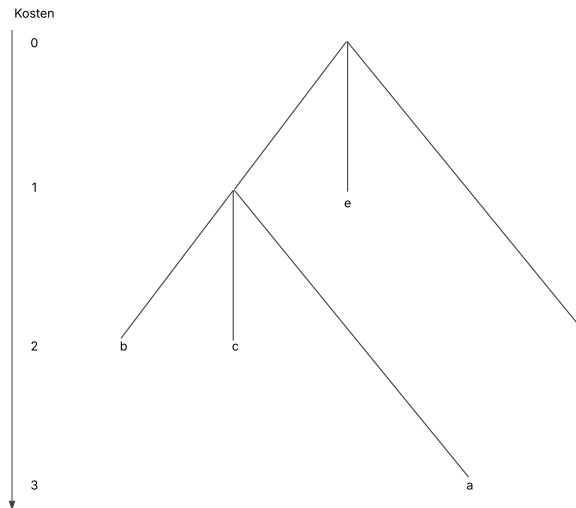


Abbildung 6: lopsided tree: Perlendurchmesser: (1; 1; 2)

bleibt unverändert: Zunächst wird ein Baum erstellt, in dem alle Blätter ungefähr die gleichen Kosten haben. Anschließend wird der Buchstabe mit den höchsten Kosten nach oben verschoben, während der Baum unten erweitert wird. Dies geschieht solange, bis kein Blatt mehr eingefügt werden kann und der beste Baum wird ausgegeben.

Wenn die Durchmesser alle gleich groß sind, gilt:

$$p_i \cdot n_i = p_i \cdot \sum_n cp_n \quad (4)$$

daher kann der zweite Algorithmus sowohl für gleiche als auch unterschiedliche Durchmesser benutzt werden.

2.3 Umsetzung

Die Lösung wurde in Python3 geschrieben. Die Eingabetexte werden eingelesen. Die Durchmesser der Perlen werden in einer Liste gespeichert und der Text in einem string. Aus dem Text werden anschließend zwei Listen erstellt: Die erste enthält die Häufigkeiten der Buchstaben in absteigender Reihenfolge. Die zweite enthält die Buchstaben entsprechend dieser Reihenfolge. Sie sehen für Beispielaufgabe *schmuck0* so aus:

1. [6, 3, 3, 2, 2, 1]
2. ['d', 'a', 's', 'f', 'g', 'h']

Um später den Buchstaben die Häufigkeit wieder richtig zuzuordnen.

2.3.1 Erstellung des Baums

Zu Beginn wird ein blanchierter Baum über die Methode `create_tree()` erstellt.

```
1 def create_tree(root: TreeNode) -> TreeNode:
```

Listing 1: Methode `create_tree`

Die Methode `create_tree` bekommt jedes potenziell neue Blatt im Baum über den Aufruf `get_all_possible_costs()` und das Blatt mit den geringsten Kosten wird dem Baum über den Aufruf `insert_new_node(obj)` hinzugefügt. Falls der Elternknoten des neuen Blatts selbst noch ein Blatt ist, entsprechen die Kosten des neuen Blatts den kombinierten Kosten für die ersten beiden Blätter. Das liegt daran, dass jeder innere Knoten mindesten zwei Kinder haben muss – daher stellen diese Kosten die effektiv neu entstehenden Kosten für den Baum dar. Das geschieht solange, bis der Baum genau so viele Blätter hat, wie es Buchstaben gibt. Seine Anzahl an Blättern erhält er über die Methode `get_size()`.

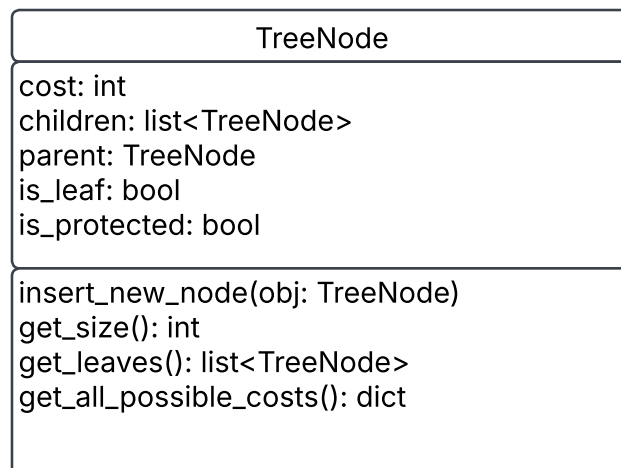


Abbildung 7: TreeNode Klasse

```
1 def get_all_possible_costs(self) -> dict:
```

Listing 2: Methode `get_all_possible_costs`

Die Methode `get_all_possible_costs` prüft, ob das `TreeNode`-Objekt noch weitere Kinder aufnehmen kann und nicht `protected` ist. Falls ja, fügt es sich selbst sowie die Kosten des potenziellen neuen Kindes einem Dictionary hinzu. Anschließend wird die Methode rekursiv für alle bestehenden Kinder aufgerufen, so dass auch deren mögliche Erweiterungen im Dictionary erfasst werden.

```
1 def insert_new_node(self, obj: TreeNode):
```

Listing 3: Methode `insert_new_node`

Die Methode `insert_new_node` prüft, ob das aktuelle Objekt selbst das gesuchte `TreeNode` Objekt ist. Falls ja, fügt es ein neues Kind hinzu. Wenn vorher noch keine Kinder vorhanden sind, werden zwei Kinder hinzugefügt. Ansonsten wird die Methode rekursiv auf alle bestehenden Kinder angewandt.

```
1 def get_size(self) -> int:
```

Listing 4: Methode `get_size`

Die Methode `get_size` gibt den Wert 1 zurück, falls das Objekt ein Blatt ist. Ansonsten wird die Methode rekursiv bei den Kinder aufgerufen und die Summe zurückgegeben.

2.3.2 Verbesserung des Baums

Sobald ein balancierter Baum erstellt wurde, muss dieser optimiert werden. Dies geschieht über die Methode `find_best_tree`.

```
1 def find_best_tree(root: TreeNode) -> TreeNode:
```

Listing 5: Methode `find_best_tree`

Die Methode *find_best_tree* versucht, den übergebenen Baum so lang zu optimieren, bis alle Blätter als *protected* markiert sind und keine Blätter mehr hinzugefügt werden können. Dazu wird der Optimierungsschritt über den Aufruf *improve_tree(root)* wiederholt und jeweils mit der Methode *calculate_tree(root)* die neuen Kosten des Baums berechnet. Sind die neuen Kosten ein neues Minimum wird der aktuelle Baum als bester Zwischenstand gespeichert. Können keine neuen Blätter dem Baum hinzugefügt werden und es tritt ein Fehler auf, wird der bis dahin beste gefundene Baum zurückgegeben.

```
1 def improve_tree(root: TreeNode) -> TreeNode:
```

Listing 6: Methode *improve_tree*

Zu Beginn holt sich die Methode *improve_tree* alle Blätter im Baum über den Aufruf *get_leaves()*. Diese Liste wird dann aufsteigend sortiert und die Kosten für jedes Element können mit der Gleichung (3) berechnet werden. Der Elternknoten des Blatts mit den höchsten Kosten wird daraufhin selbst zu einem Blatt. Dafür werden die Attribute des Elternknoten

```
1 is_leaf = True
  is_protected = True
3 children = []
```

gesetzt. Um die fehlenden Blätter wieder hinzuzufügen wird noch die Methode 1 (*create_tree*) aufgerufen.

```
1 def calculate_tree(root: TreeNode) -> TreeNode:
```

Listing 7: Methode *calculate_tree*

Die Methode *calculate_tree* berechnet die gesamten Kosten des Baums. Dafür holt sie sich auch über den Aufruf *get_leaves()* alle Blätter im Baum. Die Liste an Blättern wird dann nach den Kosten aufsteigend sortiert und jedes Element wird wie in der Gleichung (3) berechnet und zusammenaddiert.

2.4 Laufzeit und Diskussion

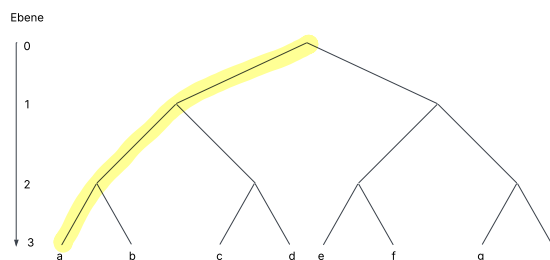


Abbildung 8: Balancierter Baum, Markierung der richtigen Knoten

2.4.1 Analyse des Algorithmus

Der Algorithmus endet, wenn keine Knoten mehr eingefügt werden können. Dies ist spätestens gegeben, wenn der Baum maximal ausgeartet ist (Abbildung 5). Allgemein kann jeder Knoten maximal $r - 1$ Mal verschoben werden, bis er komplett auf der linken Seite ist. Dabei ist r die Anzahl an verschiedenen Perlen. Daraus folgt, dass die maximale Anzahl an Iterationen

$$I < (r - 1) \cdot \text{Anzahl falscher Knoten} \quad (5)$$

ist, bis zum maximal ausgearteten Baum. Falsche Knoten sind dabei innere Knoten, die beim ausbalancierten Baum noch nicht auf der richtigen Stelle sind, also alle bis auf die äußerst linken Knoten. (Abbildung 8)

Generell gilt, um die maximale Anzahl an inneren Knoten zu erhalten, sollte jeder Knoten möglichst wenig Kinder besitzen. Wie zuvor festgelegt, hat jeder innere Knoten mindestens zwei Kinder. Daraus folgt, dass die maximale Anzahl an inneren Knoten erreicht wird, wenn der Baum ein Binärbaum ist und kann durch

$$\text{innere Knoten} = w - 1 \quad (6)$$

berechnet werden. Dabei ist w die Anzahl an Blättern/Codewörter. Auf jeder Ebene existiert ein richtiger innere Knoten, deren Anzahl minimal mit

$$\text{richtige innere Knoten} = \log_{r-1} w \quad (7)$$

abgeschätzt werden kann.

Daher ergibt sich

$$I < (r - 1) \cdot (w - 1 - \log_{r-1} w) \quad (8)$$

Dies bedeutet eine Laufzeit von $O(r \cdot w)$.

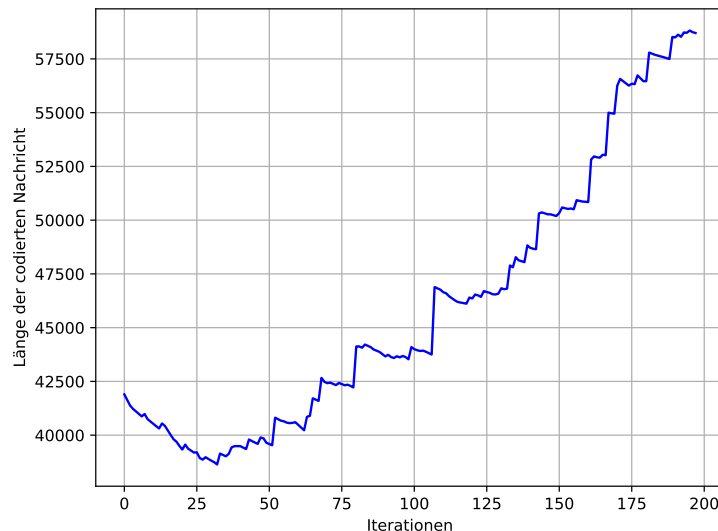


Abbildung 9: Länge der codierten Nachricht über mehrere Iterationen

Abbildung 9 zeigt wie die Länge der codierten Nachricht der Beispielaufgabe *schmuck9* sich über die Iterationen hinweg verändert. Das Minimum wird bereits in der ersten Hälfte der Iterationen erreicht.

2.4.2 Stand der Forschung

Die vorliegende Fragestellung ist seit den 60iger-Jahren in der theoretischen Informatik bekannt. Dabei lieferte Huffman eine Lösung für den Spezialfall, von zwei Quellsymbolen. [3]

Für unseren Fall von mehr als zwei Quellsymbolen mit unterschiedlichen Kosten, entwickelte bereits Richard Karp 1961 [4] eine Lösung. Diese wurde 1998 von Mordecai J. Golin und Günter Rote verbessert und ist bis heute das schnellste Verfahren, um auf eine exakte Lösung zu kommen.

”The minimum-cost prefix-free code for n words can be computed in $O(n^{C+2})$ time and $O(n^{C+1})$ space, if the costs of the letters are integers between 1 and C .”

Für die Beispielaufgabe *schmuck5.txt* hat der Algorithmus eine Laufzeit 34^8 ($n=34$; $C=6$) Iterationen. Diese Art von Algorithmus kann wegen seiner langen Laufzeit nicht für die vorgegebenen Beispielaufgabe verwendet werden. [1]

Um eine polynominelle Laufzeit zu erreichen, entwickelten Mordecai J. Golin, Claire Mathieu und Neal E. Young 2012 ein Approximationsverfahren.

Dieser Algorithmus basiert auf dem PTAS-Verfahren, wobei zuerst ein präfixfreier Code bis zur einer

bestimmten Tiefe τ erstellt wird und dann die restlichen Codewörter durch ein Rundungsverfahren ergänzt werden. Dabei ist es zentral, dass die Wahrscheinlichkeitsverteilung der zu kodierenden Codewörter möglichst in großen Gruppen mit gleich großer Wahrscheinlichkeit eingeordnet werden können. In den Beispielen, vor allem in Beispiel 9 ist dies nicht gegeben, weshalb das Bestimmen des Vorbaums über 9^{24} Schritte benötigt. Somit ist dieses Verfahren für die gewählten Beispiele ebenfalls nicht verwendbar. [2]

2.4.3 Fazit

Für die gegebenen Fragestellung gibt es bislang weder ein Beweis, dass sie NP-schwer ist, als auch keinen Gegenbeweis, dass sie es nicht ist. Aktuell bekannte Verfahren besitzen eine exponentielle Laufzeit. [1] Mein vorgestellter Algorithmus findet nicht die beste Lösung für das gegebene Problem. Er liefert jedoch in polynomieller Laufzeit eine gute Approximation und ist daher eine gute Abwägung zwischen Laufzeit und Genauigkeit. Somit wird nicht beantwortet, ob die Fragestellung NP-schwer ist.

2.5 Vergleich zur Lösung

In der Musterlösung werden drei Lösungen vorgeschlagen:

1. ein Greedy-Algorithmus basierend auf Huffman [3]
2. ein ILP-Ansatz nach Richard Karp [4] und
3. die Erweiterung von Mordecai J. Golin und Günter Rote [1]

Alle drei Ansätze waren mir bei Abgabe meiner Arbeit bekannt (siehe Abschnitt 2.4.3). Trotzdem entschied ich mich dagegen eines dieser Verfahren zu implementieren, vorallem aufgrund der hohen Laufzeit, und entwickelte stattdessen meine eigenes Verfahren. Mein Algorithmus hat dabei eine durchschnittliche Abweichung von 1,27% vom ideal Ergebnis, besitzt aber dafür eine sehr viel kürzere Laufzeit. (Todo: Hinweis auf die Lösungen) Für mein gutes Ergebnis erhielt ich daher auch einen Pluspunkt. Da ich außerdem zu allen Teilaspekten der Aufgabe Stellung nahm und keine Punkte abgezogen wurden, erreichte ich insgesamt 21 Punkte für diese Aufgabe.

3 Aufgabe 2: Simultane Labyrinth

3.1 Einleitung

In dieser Aufgabe möchten drei Freunde, Anton, Bea und Chris, zwei Labyrinth gleichzeitig durchlaufen. Beide Labyrinth haben die selben rechteckigen Maße (n, m) . Zwei starteten jeweils in einem Labyrinth bei der Position $(0, 0)$ und müssen das Ziel (n, m) erreichen, also von der linken oberen Ecke zur rechten unteren Ecke. Ein dritter gibt jeweils beiden immer dieselbe nächste Schrittanweisung. Ist der nächste Schritt in einem Labyrinth blockiert, bleibt die Person stehen. Gesucht ist eine möglichst kurze Abfolge von Schritten, die beide Labyrinth mit den selben Befehlen löst. Bei dieser Aufgabe wird zusätzlich zwischen zwei Varianten unterschieden: in der ersten Teilaufgabe handelt es sich um Labyrinth ohne Grube und in der zweiten Teilaufgabe um Labyrinth mit Grube. Fällt ein Spieler in eine Grube, wird seine Position auf die Startposition $(0, 0)$ zurückgesetzt. Ich bearbeite zuerst die Variante ohne die Gruben und erweitere sie anschließend auf Labyrinth mit Gruben.

3.2 Lösungsidee

Zu Beginn wird für jedes Labyrinth eine Matrix erstellt (Abbildung 10), in der für jede Position ein Tuple aus einem Kostenwert und einer Richtung gespeichert wird. Die gespeicherte Richtung zeigt den optimalen Folgeschritt zum Ziel und ist dargestellt als

Code	Richtung
0	Rechts
1	Links
2	Oben
3	Unten

Tabelle 1: Codierung der Richtungen im Labyrinth

Der Kostenwert gibt an, wie viele Schritte bis zum Ziel benötigt werden.

Ein Labyrinth mit dazugehöriger Matrix könnte so aussehen:

13;3	14;1	5;0	4;0	3;3
12;3	13;1	14;1	5;2	2;3
11;3	8;0	7;0	6;2	1;3
10;0	9;2	8;2	7;2	0;null

Abbildung 10: Labyrinth und Matrix

3.2.1 Erstellung der Matrix

Zu Beginn wird eine Matrix erstellt, deren Werte für alle Positionen zunächst leer sind. Ausgehend vom Zielpunkt durchläuft der Algorithmus rekursiv alle erreichbaren Nachbarpositionen. Für jeden Nachbarn wird in der Matrix ein Kostenwert eingetragen, der um eins höher ist als der Kostenwert des aktuellen Punkts, sowie die Richtung, aus der der Nachbar erreicht wurde.

3.2.2 Lösen der Labyrinth

Der Algorithmus funktioniert so, dass er jeweils den momentan besten Weg aus der Liste der bisherigen Wege entnimmt und diesen um einen weiteren Schritt erweitert. Den nächsten Schritt ermittelt er, indem er in der Matrix prüft, welchen Schritt die beiden Labyrinth für ihre jeweilige Position vorschlagen. Dieser Schritt wird an die bisherige Schrittfolge angehängt. Dabei entstehen in der Regel zwei unterschiedliche Wege. Beide werden zur Liste der bisherigen Wege hinzugefügt. Zur Auswertung der bisherigen Wege wird diese Rechnung angewandt:

$$\frac{c_b - c_a}{n} \quad (9)$$

wobei gilt:

- c_b sind die summierten Kosten von beiden Startpositionen
- c_a sind die summierten Kosten von beiden aktuellen Positionen
- n ist die Anzahl der benötigten Schritte

Der Bruch beschreibt also die durchschnittliche Einsparung von Kosten pro Schritt. Je höher der Wert ist, desto effizienter ist der bisherige Pfad. Dabei ist 2 der maximal erreichbare Wert.

Der Weg mit der höchsten Einsparung gilt dabei als momentan bester Weg. Nach jeder Iteration wird das aktuelle Positionspaar $((x_1, y_1), (x_2, y_2))$ in einem Dictionary gespeichert, um zu verhindern, dass der Algorithmus das exakt selbe Positionspaar noch einmal besucht. Zusätzlich gibt es auch die Begrenzung l , die festlegt, wie viele unterschiedliche Wege, sortiert nach ihren Kosten, in der Liste gespeichert werden dürfen. Diese Begrenzung beeinflusst maßgeblich die Laufzeit und Genauigkeit des Algorithmus. In der Liste dürfen die Wege unterschiedliche Werte für n haben; wichtig ist lediglich, dass ihre Kosten (Gleichung (9)) gering sind. Der gesamte Ablauf wiederholt sich solange, bis beide im Ziel sind.

3.2.3 Gruben

Gruben werden sowohl in der Kostenmatrix als auch beim Lösen wie normale Felder gesehen. Berührt jedoch einer der beiden eine Grube, wird seine Position auf die jeweilige Startposition zurückgesetzt. Dadurch steigen die Gesamtkosten des Wegs so weit an, dass dieser wegen der Begrenzung l in der Regel aus der Liste der potenziellen Wege entfernt wird.

3.3 Umsetzung

Die Lösung wurde in Python3 geschrieben. Die Eingabetexte werden eingelesen. Für beide Labyrinth werden die horizontalen und vertikalen Wände jeweils in separate Listen gespeichert. Auch die Positionen der Gruben werden für jedes Labyrinth in einer eigenen Liste erfasst.

3.3.1 Erstellung der Matrix

Zur Erstellung der Matrix wird die Klasse *cost* benutzt.

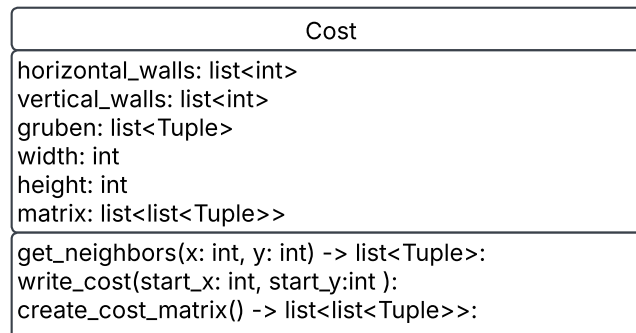


Abbildung 11: Cost Klasse

Für jedes Labyrinth wird ein Cost-Objekt erstellt und die Methode *cost_obj.create_cost_matrix()* aufgerufen. Das Cost-Objekt bekommt als Übergabewerte die horizontalen und vertikalen Wände, die Gruben sowie die Größe des Labyrinths.

```
1 def create_cost_matrix(self) -> list:
```

Listing 8: Methode *create_cost_matrix*

Die *create_cost_matrix* Methode erstellt zunächst eine leere Liste mit der Dimension [height][width], die als Kostenmatrix dient. Dann ruft sie die Methode *write_cost* auf, um die Kostenmatrix zu füllen.

```
1 def write_cost(self, start_x: int, start_y: int):
```

Listing 9: Methode *write_cost*

Zu Beginn der Methode *write_cost()* wird die Liste *stack* initialisiert. Sie speichert alle Felder, die noch besucht werden müssen. Jeder Eintrag in der Liste besitzt vier Werte: die x-Position, die y-Position, die aktuellen Kosten sowie die entgegengesetzte Richtung, aus welcher das Feld betreten wurde. Die Startposition wird als erstes Element hinzugefügt. Anschließend wird das erste Element der Liste entnommen und in die Kostenmatrix übertragen. Falls es schon in der Kostenmatrix ist, wird es übersprungen. Daraufhin werden alle benachbarten Felder mit der Methode *get_neighbours* ermittelt und dem *stack* hinzugefügt. Dabei werden die Kosten um eins erhöht und die jeweilige Richtung gespeichert. Das wiederholt sich solange bis der *stack* leer ist.

3.3.2 Lösen der Labyrinth

Zum Lösen der Labyrinth wird die Klasse *Solving* benutzt.

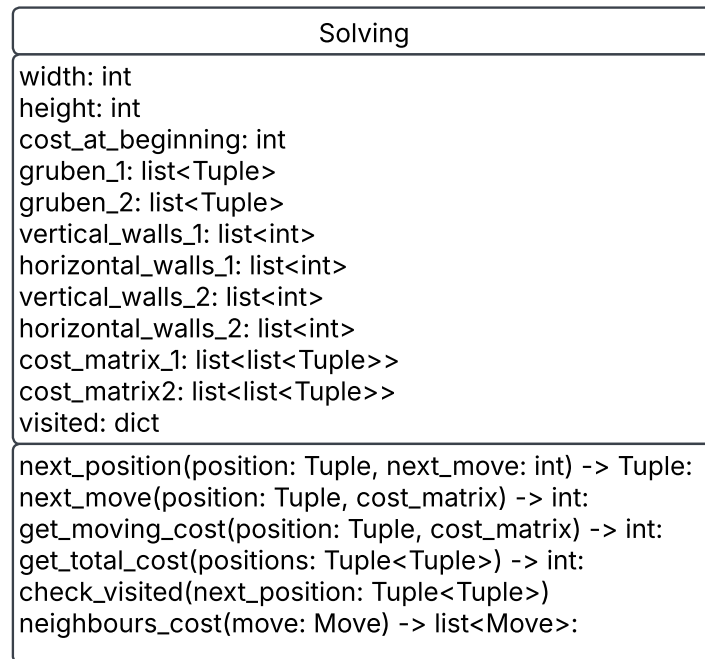


Abbildung 12: Solving Klasse

Die Wege werden mit der Klasse *Move* dargestellt.

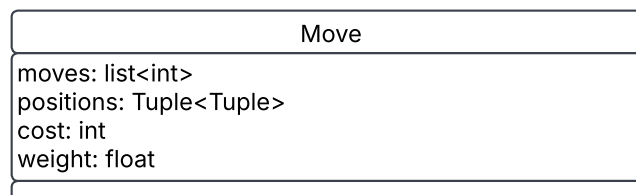


Abbildung 13: Move Klasse

Das Attribut *cost* beschreibt die aufsummierten Kosten der beiden aktuellen Positionen basierend auf den Werten aus den Kostmatrizen. Das Attribut *weight* wird gemäß der Gleichung (9) berechnet.

Zu Beginn wird ein *Solving*-Objekt initialisiert und eine leere Liste *moves* angelegt, in der alle aktuellen Wege gespeichert werden. Anschließend wird ein erstes *Move*-Objekt erzeugt, dessen Startposition $((0, 0), (0, 0))$ ist und seine *moves* leer sind. In jedem Schritt wird das erste Element aus der *moves* Liste entnommen. Dann wird mit folgendem Aufruf

```
1 new_moves = solving_obj.neighbours_cost(move)
```

eine Liste von möglichen Folgewege erzeugt. Die Liste wird mit den neuen Wegen erweitert, sortiert und auf die *l* besten Wege beschränkt.

```
1 def neighbours_cost(self, move: Move) -> list:
```

Listing 10: Methode *neighbours_cost*

Zu Beginn der Methode *neighbours_cost()* werden die beiden Positionen aus dem Attribut *positions* des übergebenen *move*-Objekt genommen. Anschließend werden mit den folgenden Aufrufen

```
1 move_1 = self.next_move(pos1, self.cost_matrix_1)
  move_2 = self.next_move(pos2, self.cost_matrix_2)
```

die jeweils nächsten möglichen Richtungen aus der Kostenmatrix bestimmt, falls das Ziel noch nicht erreicht wurde. Daraufhin werden die neuen zwei Positionspaare berechnet, wenn jeweils in die Richtungen gegangen werden würde.

```

next_position_1 = self.next_position(move.positions, move_1)
2 next_position_2 = self.next_position(move.positions, move_2)

```

Mit dem folgenden Aufruf wird anschließend überprüft, ob die beiden Positionspaare bereits besucht wurden:

```

move_1 = self.check_visited(next_position_1, len(move.moves) + 1, move_1)
2 move_2 = self.check_visited(next_position_2, len(move.moves) + 1, move_2)

```

Zum Schluss wird für jeden möglichen Zug ein *Move*-Objekt erstellt und zusammen in einer Liste übergeben. Falls einer der neuen Positionspaare bereits besucht wurde oder bereits im Ziel ist, wird der entsprechende Zug nicht übergeben.

Im folgenden ist die Methode *neighbours_cost* als Flussdiagramm dargestellt.

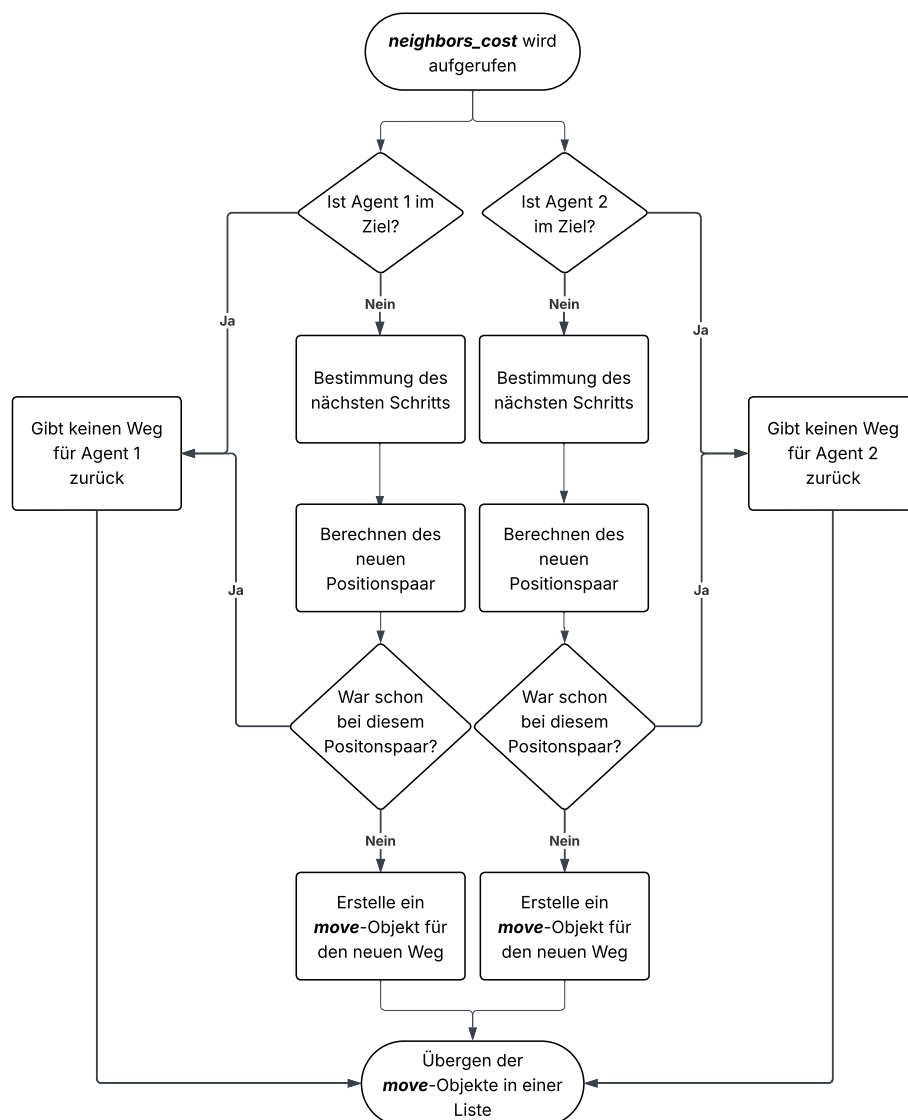


Abbildung 14: *neighbours_cost* Flussdiagramm

3.4 Laufzeit und Diskussion

3.4.1 Komplexität der Lösung

Von jeder Position aus existieren im schlimmsten Fall zwei mögliche Schritte, entweder in Richtung des Ziels von Labyrinth 1 oder in Richtung des Ziels von Labyrinth 2. Das führt zu insgesamt $2^{\text{maximale Anzahl an Schritten}}$ verschiedenen Möglichkeiten. Aufgrund dieser großen Anzahl an Möglichkeiten untersucht der Algorithmus nicht alle Pfade vollständig, sondern macht nur eine Abschätzung. Der hier vorgestellte Ansatz garantiert daher nicht unbedingt den optimalen Lösungsweg, liefert jedoch in akzeptabler Laufzeit eine gute Annäherung. Der Algorithmus ähnelt einer Tiefensuche, da er einige Wege tief verfolgt, während andere nur kurz betrachtet werden.

3.4.2 Laufzeit

Wie schon oben erwähnt beeinflusst die Begrenzung l , die Laufzeit und Genauigkeit des Algorithmus stark.

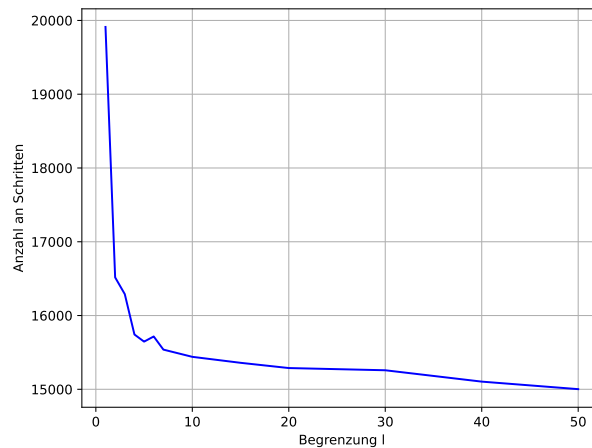


Abbildung 15: Anzahl der Schritte

Abbildung 15 zeigt die Anzahl der Schritte des gefundenen Lösungsweges in Abhängigkeit vom Parameter l .

Auffällig ist, dass der Graph mit zunehmendem l gegen die Anzahl der Schritte des optimalen Weges konvergiert. Ebenso erkennbar ist, dass er nicht streng monoton fallend ist. In einem Fall, bei dem die Steigung positiv ist, wird für ein größeren Wert von l ein schlechterer Pfad gefunden. Dies liegt daran, dass durch den höheren Wert von l neue Wege betrachtet werden. Diese erscheinen kurzfristig vorteilhafter als die bisherigen Wege und verdrängen dadurch die eigentlich besseren aus der Liste. Das führt insgesamt zu einem schlechteren Weg. Bei größeren l -Werten tritt dieser Effekt jedoch kaum noch auf, da die Liste groß genug ist, um alle relevanten Pfade zu behalten. Der Einfluss dieser Monotonieveränderungen wird daher mit wachsendem l vernachlässigbar.

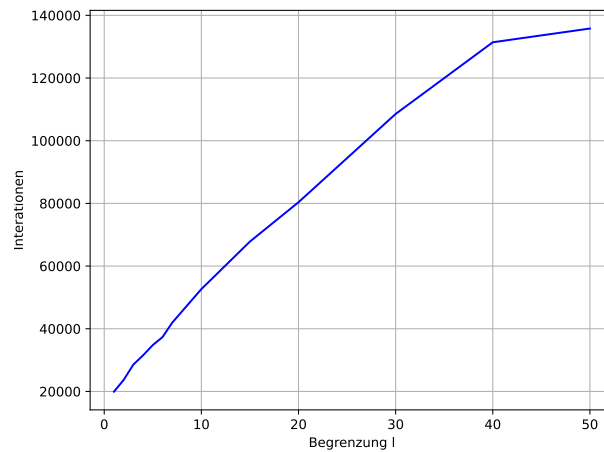


Abbildung 16: maximale Iterationen gegenüber tatsächlichen Iterationen

Abbildung 16 stellt die Anzahl an Iterationen dar, die zur Bestimmung des Lösungswegs benötigt werden in Abhängigkeit vom Parameter l .

Der Graph steigt mit wachsendem l stark an.

Allgemein kann die maximale Anzahl an Iterationen in Abhängigkeit von l wie folgt berechnet werden.

$$l \cdot (2c_1 + c_2) \quad (10)$$

wobei:

- c_1 die Kosten vom Startpunkt aus der Kostenmatrix 1 sind
- c_2 die Kosten vom Startpunkt aus der Kostenmatrix 2 sind

Der Faktor $2c_1 + c_2$ beschreibt die maximale Anzahl an Schritten, aus denen ein möglicher Lösungsweg bestehen kann. Zunächst erreicht Agent1 das Ziel mit c_1 Schritten. Anschließend läuft Agent2 denselben Weg zurück in ebenfalls c_1 Schritten und geht dann selber ins Ziel mit c_2 Schritten. Im schlechtesten Fall geht er für jeden Eintrag in der Liste, also l mal, die maximale Anzahl an Schritten. Daher ergibt sich die Gleichung (10), dies bedeutet, es ergibt sich eine Laufzeit von $o(l)$ ergibt. Praktisch gesehen ist die Laufzeit aber sehr viel kleiner als die maximale Laufzeit.

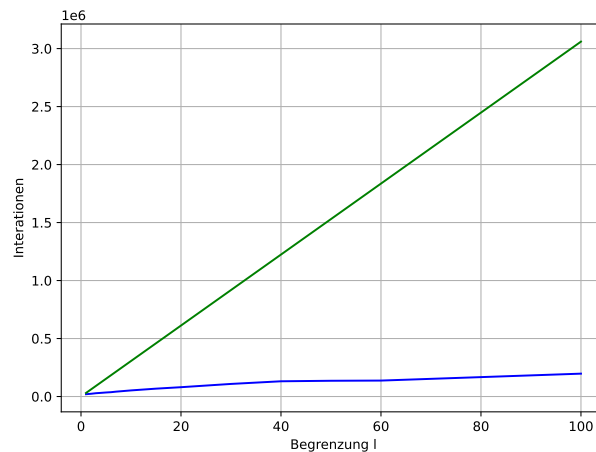


Abbildung 17: maximale Iterationen gegenüber tatsächliche Iterationen

Der grüne Graph zeigt die maximale Anzahl an Iterationen und der blaue die tatsächliche Anzahl an Iterationen.

3.4.3 Fazit

Die Werte von l stehen in einem linearen Zusammenhang mit der Anzahl der Iterationen, während die Anzahl der Schritte bei größeren l -Werten gegen das optimale Ergebnis konvergieren. Je größer der Wert

von l , desto mehr Iteration werden durchgeführt - und desto besser wird das Ergebnis. Die Genauigkeit lässt sich anhand des Parameters l jedoch nicht allgemein bestimmen, da sie stark von der Struktur des jeweiligen Labyrinths abhängt.

3.5 Vergleich zur Lösung

Die Musterlösung stellt zwei mögliche Lösungsstrategien vor:

1. Breitensuche auf zwei Labyrinth: Diese Variante wird in der Musterlösung aufgrund ihrer hohen Laufzeit als nicht praktikabel eingestuft.
2. A-Star Algorithmus mit Priority Queue: Diese Option ähnelt stark meiner eigenen Idee. Wie in meiner Kostenmatrix wird auch hier für jede Position die Distanz zum Ziel berechnet. Unterschied liegt vor allem in einem effizienteren Warteschlangenmodell, das in der Musterlösung verwendet wird. Insgesamt lagen meine Ergebnisse, aus geschlossen von Testfall 8, bei 1,1% Abweichung vom optimalen Wert. Bei Testfall 8 fiel mein Ergebnis etwa 3-mal so groß aus, weshalb mir 2 Punkte im Kriterium "Verfahren mit guten Ergebnissen" abgezogen wurden. Außerdem war meine Überlegen zur Laufzeit des Verfahrens nicht ausreichend detailliert, was zu einem weiteren Punktabzug führte. Insgesamt erhielt ich für die Aufgabe 17 von 20 Punkten.

4 Fazit

- was nehme ich am Ende mit
- wie gut war ich im Vergleich
- was habe ich gelernt
- wie gut war das alles jetzt

5 Anhang

5.1 Beispiele

5.2 Quellcode

5.3 Quellen

- [1] Mordecai J. Golin and Günther Rote, *A Dynamic Programming Algorithm for Constructing Optimal Prefix-Free Codes with Unequal Letter Costs*, IEEE Transactions on Information Theory 44, no. 5, (September 1998)
- [2] Mordecai J. Golin, Claire Mathieu und Neal E. Young, *HUFFMAN CODING WITH LETTER COSTS: A LINEAR-TIME APPROXIMATION SCHEME*, 2012 Society for Industrial and Applied Mathematics Vol. 41, No. 3, pp. 684–713
- [3] Prof. Justus Piater, *Algorithmen und Datenstrukturen*, 29.05.2024 Universität Innsbruck, Seite 4-13
- [4] R. Karp, *Minimum-redundancy coding for the discrete noiseless channel*, IRE Transactions on Information Theory, vol. 7, no. 1, January 1961